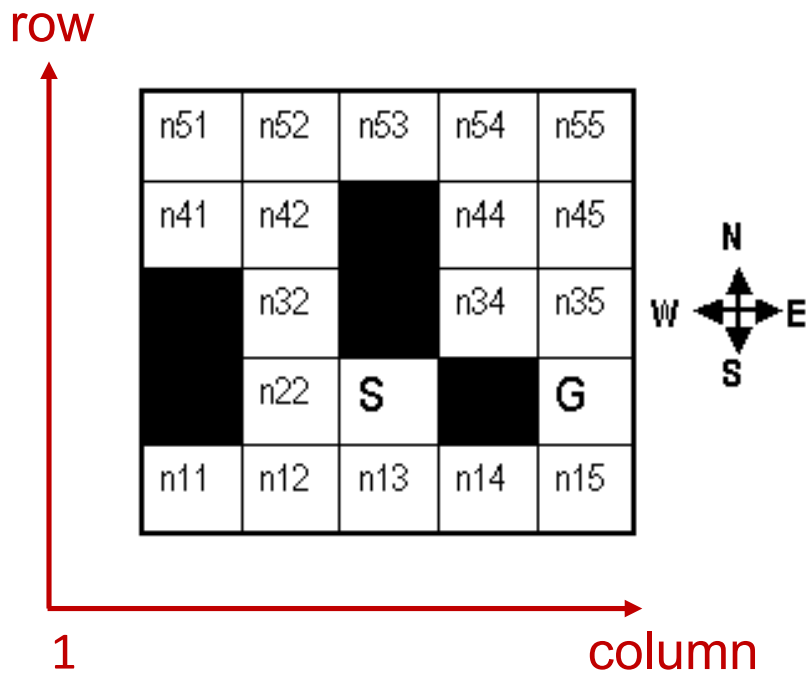
TUTORIAL#2: MAZE

Problem definition

We are trying to find a path from **S** to **G** in the following maze:

- **S** – start state
- **G** – goal state
- **Black** squares – blocked cells
- All squares, except for the start and goal states, are labelled, with row and column numbers.
  - For example, n54 is the cell in the fifth row, fourth column
- The graph **states** are the squares of the maze.



n.h.reyes@massey.ac.nz

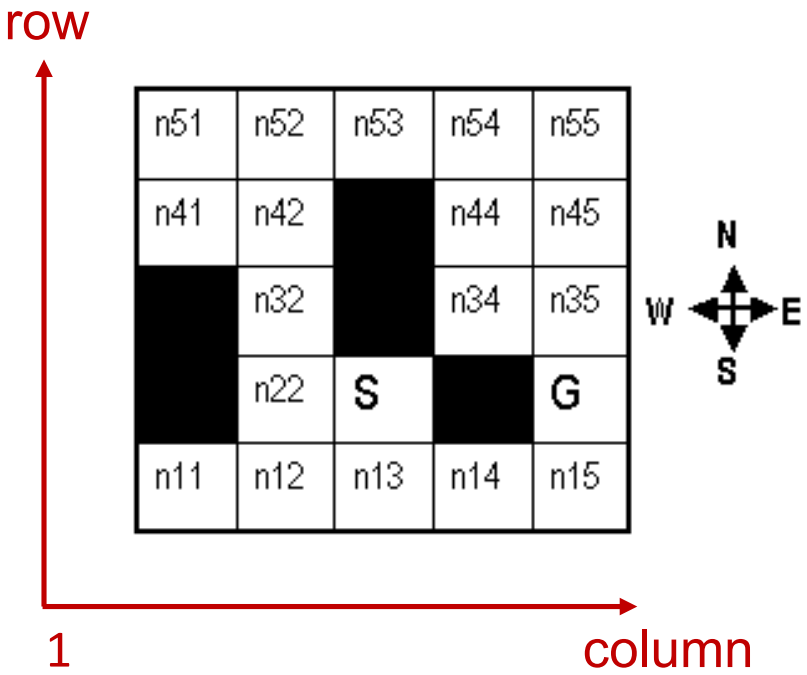
Reference: MIT OpenCourseWare

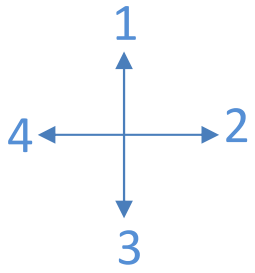
TUTORIAL#2: MAZE

Problem definition

We are trying to find a path from **S** to **G** in the following maze:

- There are **links** along the cardinal compass directions to neighbouring squares (that are not blocked). That is, diagonal moves are not permitted, nor are moves into or across a black square.
- The **successors** of a state are to be generated in a FIXED order to be drawn from left to right, namely N, E, S, W (Up, Right, Down, Left).

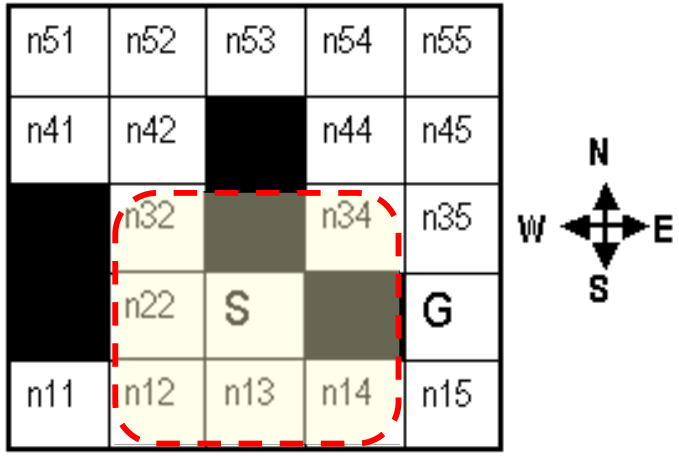




4-connected gridworld

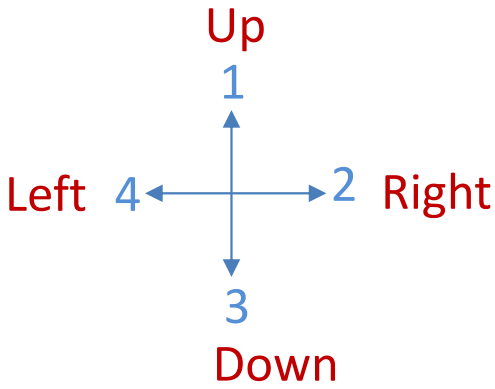
TUTORIAL#2: MAZE

Search Tree



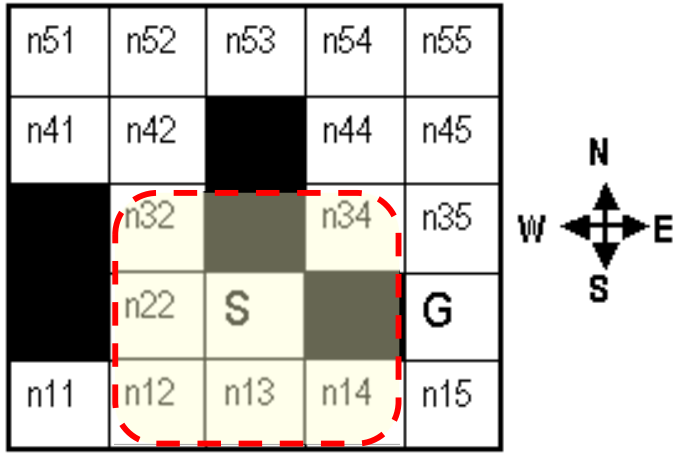
4-connected gridworld

- Start exploring the maze from S, considering the given successor generator ordering {up, right, down, left}.

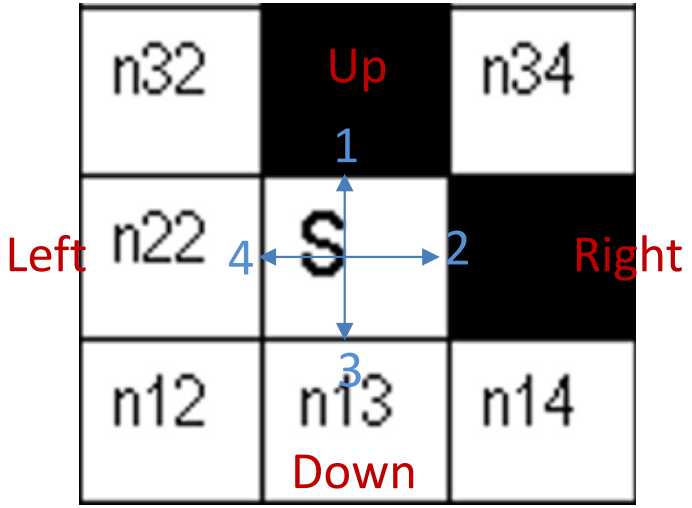


TUTORIAL#2: MAZE

Search Tree



4-connected gridworld



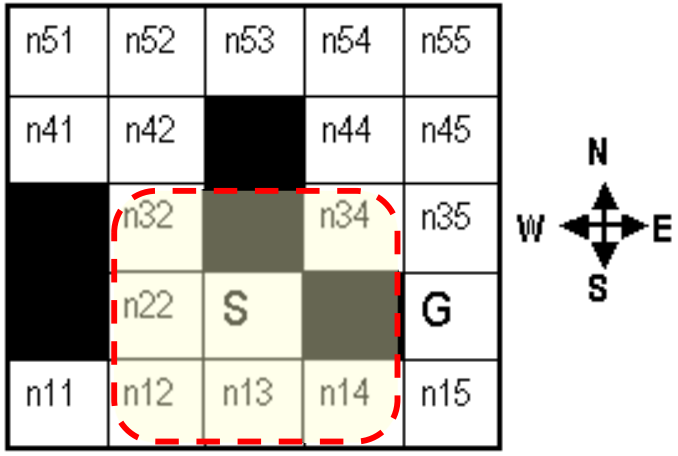
- From S, we cannot move up or right, as the cells in those neighbouring positions are all blocked. We can only move down or left.

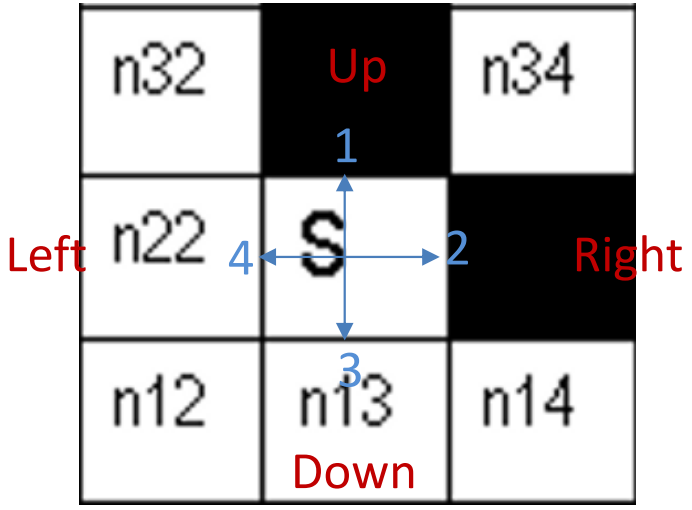
TUTORIAL#2: MAZE

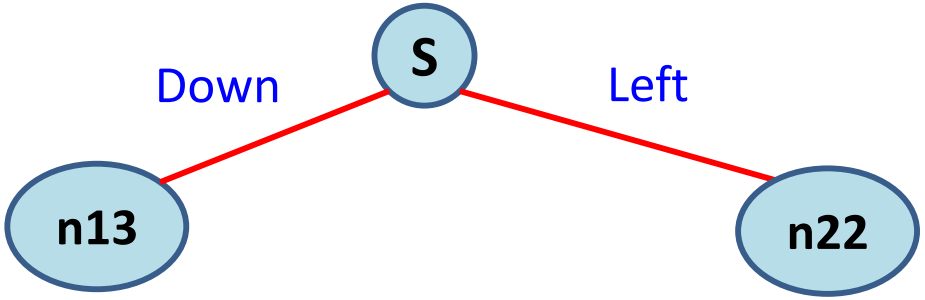
Search Tree

We are trying to find a path from **S** to **G** in the following maze:

4-connected gridworld







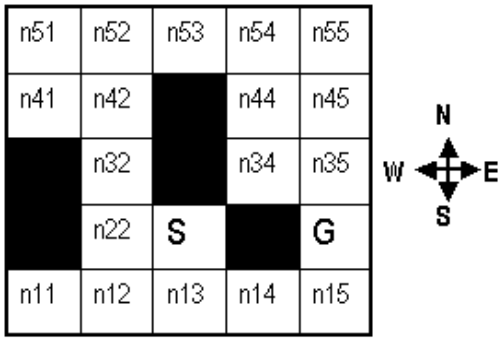
When generating a tree, remember to avoid local loops.

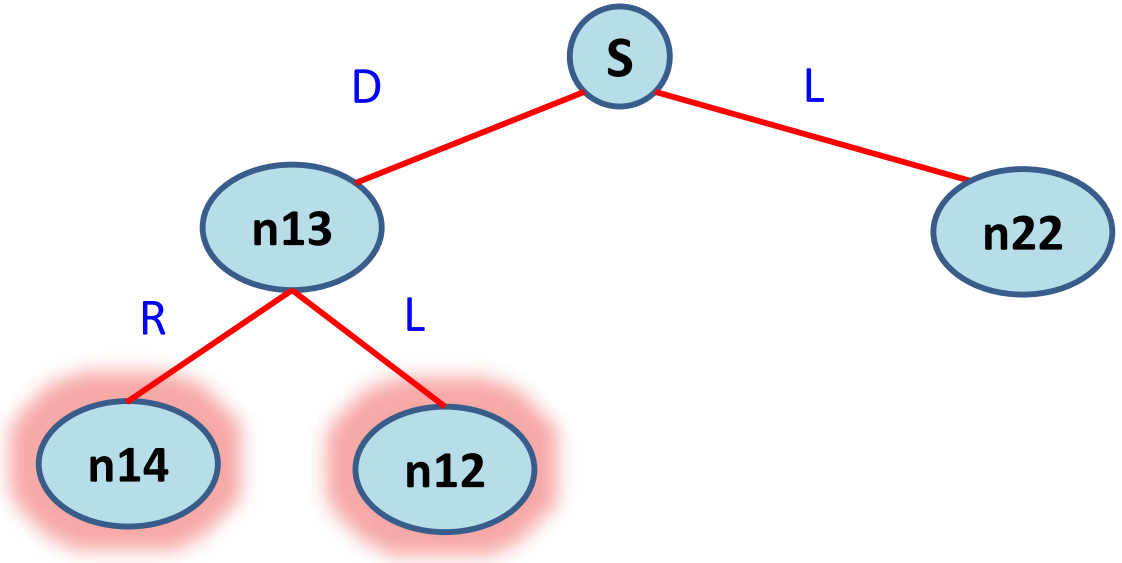
TUTORIAL#2: MAZE

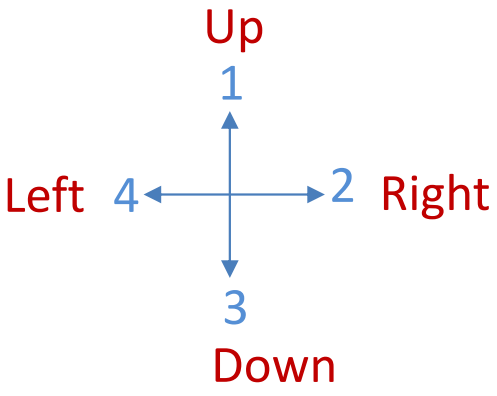
Search Tree

We are trying to find a path from **S** to **G** in the following maze:

4-connected gridworld







Extending the node n13, we can see that we can reach {S, n14 and n12}. However, we don't want to go back to S. Therefore, we are extending it to n14 and n12.

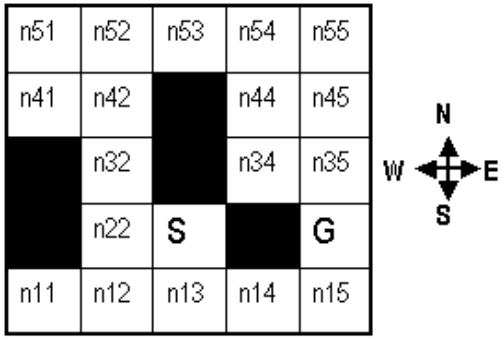
When generating a tree, remember to avoid local loops.

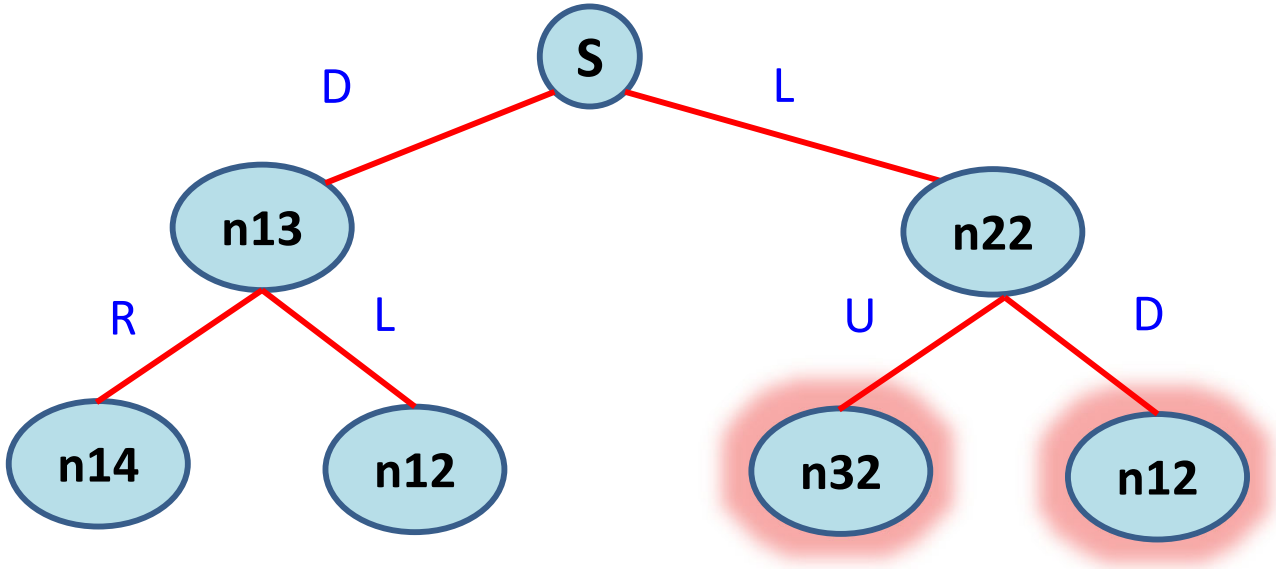
TUTORIAL#2: MAZE

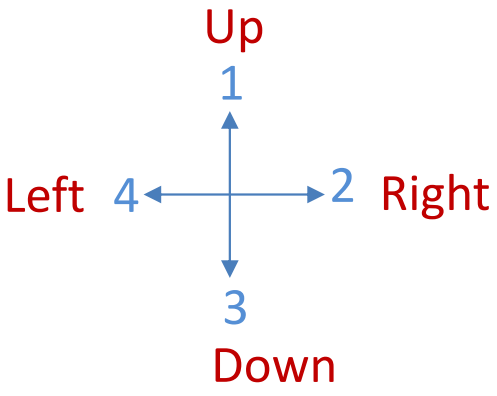
Search Tree

We are trying to find a path from **S** to **G** in the following maze:

4-connected gridworld





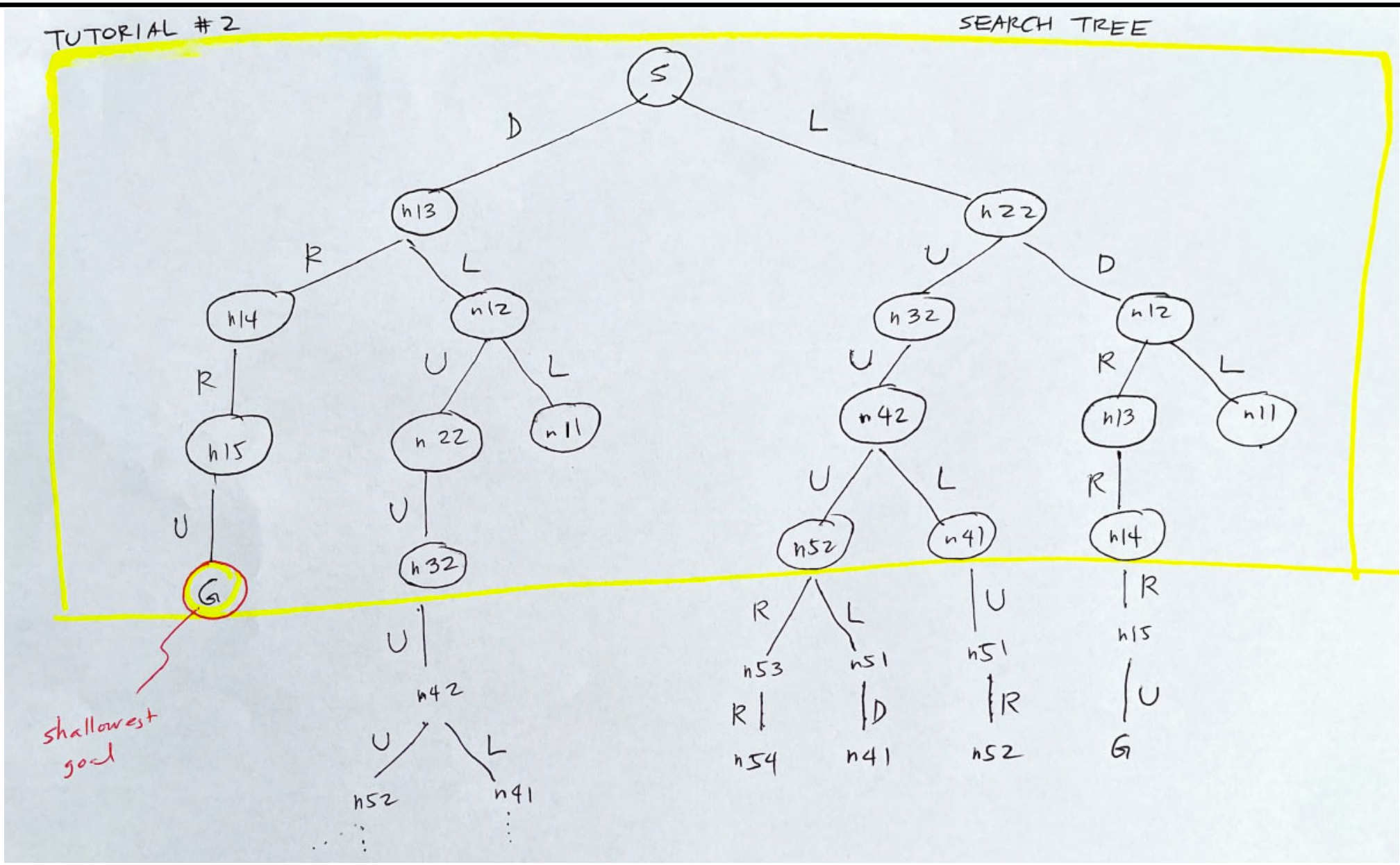


Extending the node n22, we can see that we can reach {n32,S and n12}. However, we don't want to go back to S. Therefore, we are extending it to n32 and n12.

When generating a tree, remember to avoid local loops.

TUTORIAL#2: MAZE

Search Tree

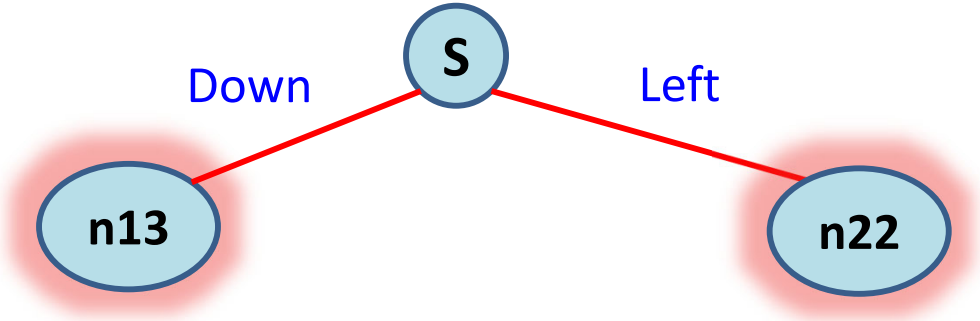


SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(n13,S) (n22,S)	S,n13,n22

Search Tree

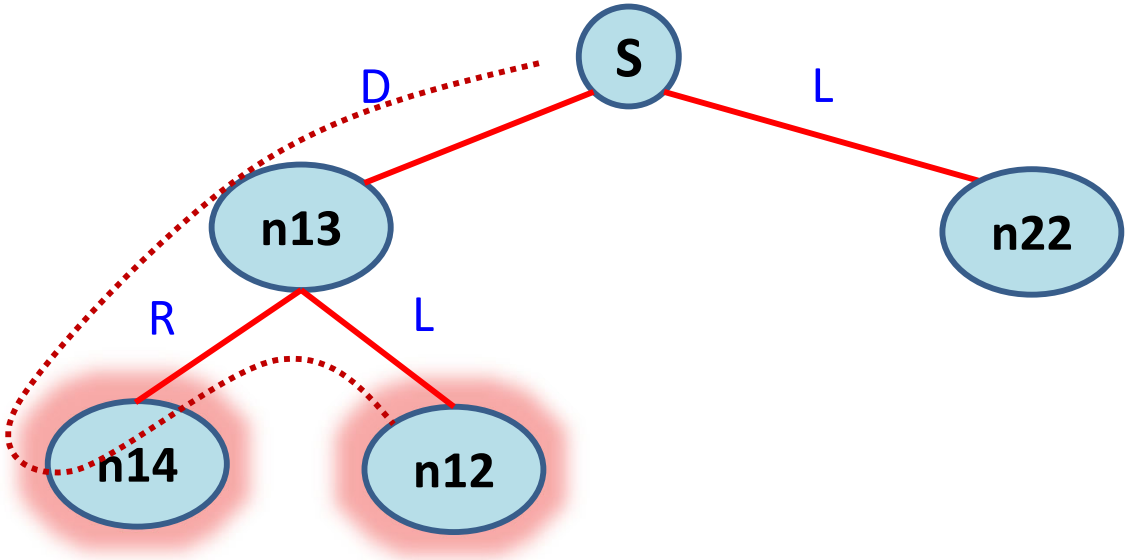


Pick **first** element of Q; Add path extensions to **end** of Q.

SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(n13,S) (n22,S)	S,n13,n22
2	(n13,S)	(n22,S) (n14,n13,S) (n12,n13,S)	“, n14, n12



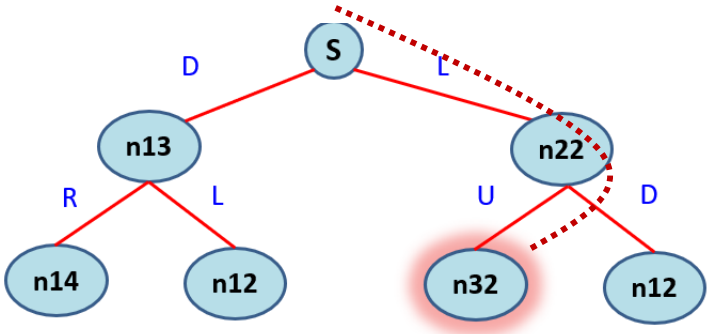
Pick **first** element of Q; Add path extensions to **end** of Q.

SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(n13,S) (n22,S)	S,n13,n22
2	(n13,S)	(n22,S) (n14,n13,S) (n12,n13,S)	“, n14, n12
3	(n22,S)	(n14,n13,S) (n12,n13,S) (n32,n22,S)	“, n32

We are skipping the path extension to **n12**, as we have visited that node previously already.



Pick **first** element of Q; Add path extensions to **end** of Q.

Breadth-First Search (with a Visited List)

Step	Dequeued	Enqueued	Visited List
0		~~(s)~~	s
1	(s)	~~(n13, s)~~ (n22, s)	s, n13, n22
2	(n13, s)	~~(n22, s)~~ (n14, n13, s) (n12, n13, s)	" n14, n12
3	(n22, s)	~~(n14, n13, s)~~ (n12, n13, s) (n32, n22, s)	" n32
4	(n14, n13, s)	~~(n12, n13, s)~~ (n32, n22, s) (n15, n14, n13, s)	" n15
5	(n12, n13, s)	~~(n32, n22, s)~~ (n15, n14, n13, s) (n11, n12, n13, s)	" n11
6	(n32, n22, s)	~~(n15, n14, n13, s)~~ (n11, n12, n13, s) (n42, n32, n22, s)	" n42
7	(n15, n14, n13, s)	~~(n11, n12, n13, s)~~ (n42, n32, n22, s) (G, n15, n14, n13, s)	" G
8	(n11, n12, n13, s)	~~(n42, n32, n22, s)~~ (G, n15, n14, n13, s)	"
9	(n42, n32, n22, s)	~~(G, n15, n14, n13, s)~~ (n52, n42, n32, n22, s) (n41, n42, n32, n22, s)	" n52, n41
10	(G, n15, n14, n13, s)		

Breadth-First Search (with a Visited List)

Step	Dequeued	Enqueued
∅		(S)
1	(S)	(n13, S) (n22, S)
2	(n13, S)	(n22, S) (n14, n13, S) (n12, n13, S)
3	(n22, S)	(n14, n13, S) (n12, n13, S) (n32, n22, S)
4	(n14, n13, S)	(n12, n13, S) (n32, n22, S) (n15, n14, n13, S)
5	(n12, n13, S)	(n32, n22, S) (n15, n14, n13, S) (n11, n12, n13, S)
6	(n32, n22, S)	(n15, n14, n13, S) (n11, n12, n13, S) (n42, n32, n22, S)
7	(n15, n14, n13, S)	(n11, n12, n13, S) (n42, n32, n22, S) (G, n15, n14, n13, S)
8	(n11, n12, n13, S)	(n42, n32, n22, S) (G, n15, n14, n13, S)
9	(n42, n32, n22, S)	(G, n15, n14, n13, S) (n52, n42, n32, n22, S) (n41, n42, n32, n22, S)
10	(G, n15, n14, n13, S)	

Sequence of state expansions = S-n13-n22-n14-n12-n32-n15-n11-n42-G

Path = S-n13-n14-n15-G

Path length = 4

Max Q Length = 3

Number of state expansions = 10

Breadth- First Search (with a Visited List)

Step	Dequeued	Enqueued	Visited List
0		(S)	
1	(S)	(n13, S) (n22, S)	S
2	(n13, S)	(n22, S) (n14, n13, S) (n12, n13, S)	S, n13, n22
3	(n22, S)	(n14, n13, S) (n12, n13, S) (n32, n22, S)	" n14, n12
4	(n14, n13, S)	(n12, n13, S) (n32	" n32
5	(n12, n13, S)	(n32, n22, S) (	
6	(n32, n22, S)	(n15, n14, n13, S) (	
7	(n15, n14, n13, S)	(n11, n12, n13, S) (	
8	(n11, n12, n13, S)	(n42, n32, n22	
9	(n42, n32, n22, S)	(G, n1	
10	(G, n15, n14, n13, S)		

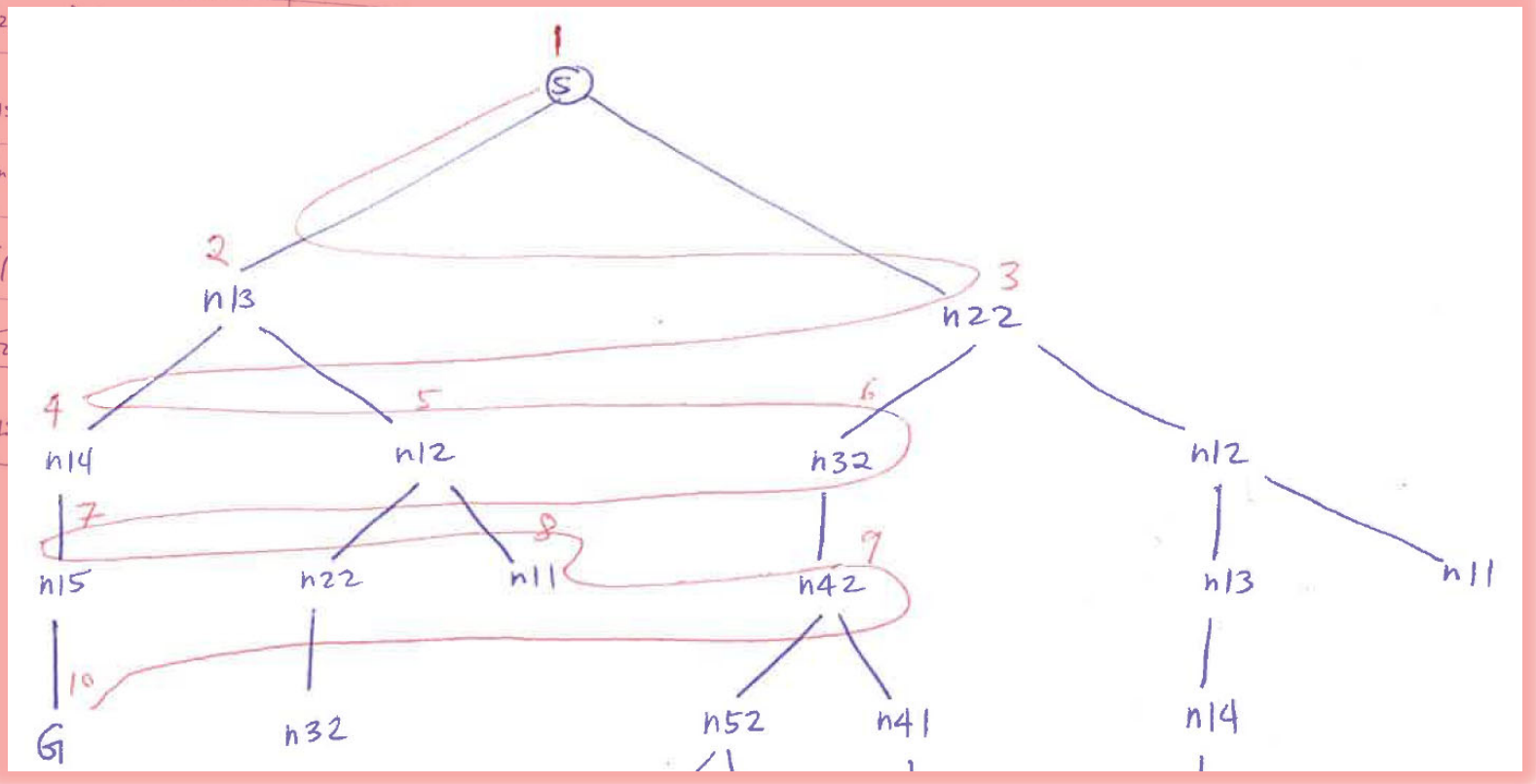
Sequence of state expansions = S-n13-n22-n14-n12-n32-n15-n11-n42-G

Path = S-n13-n14-n15-G

Path length = 4

Max Q Length = 3

Number of state expansions = 10



Tracing the sequence of state expansions shows us that the algorithm traverses the search tree following the breadth-first approach indeed.

SEARCH

Breadth-First Search with Visited List

Characterisation:

Sequence of state expansions = S-n13-n22-n14-n12-n32-n15-n11-n42-G

Path = S-n13-n14-n15-G

Path length = 4

Max Q Length = 3

Number of state expansions = 10